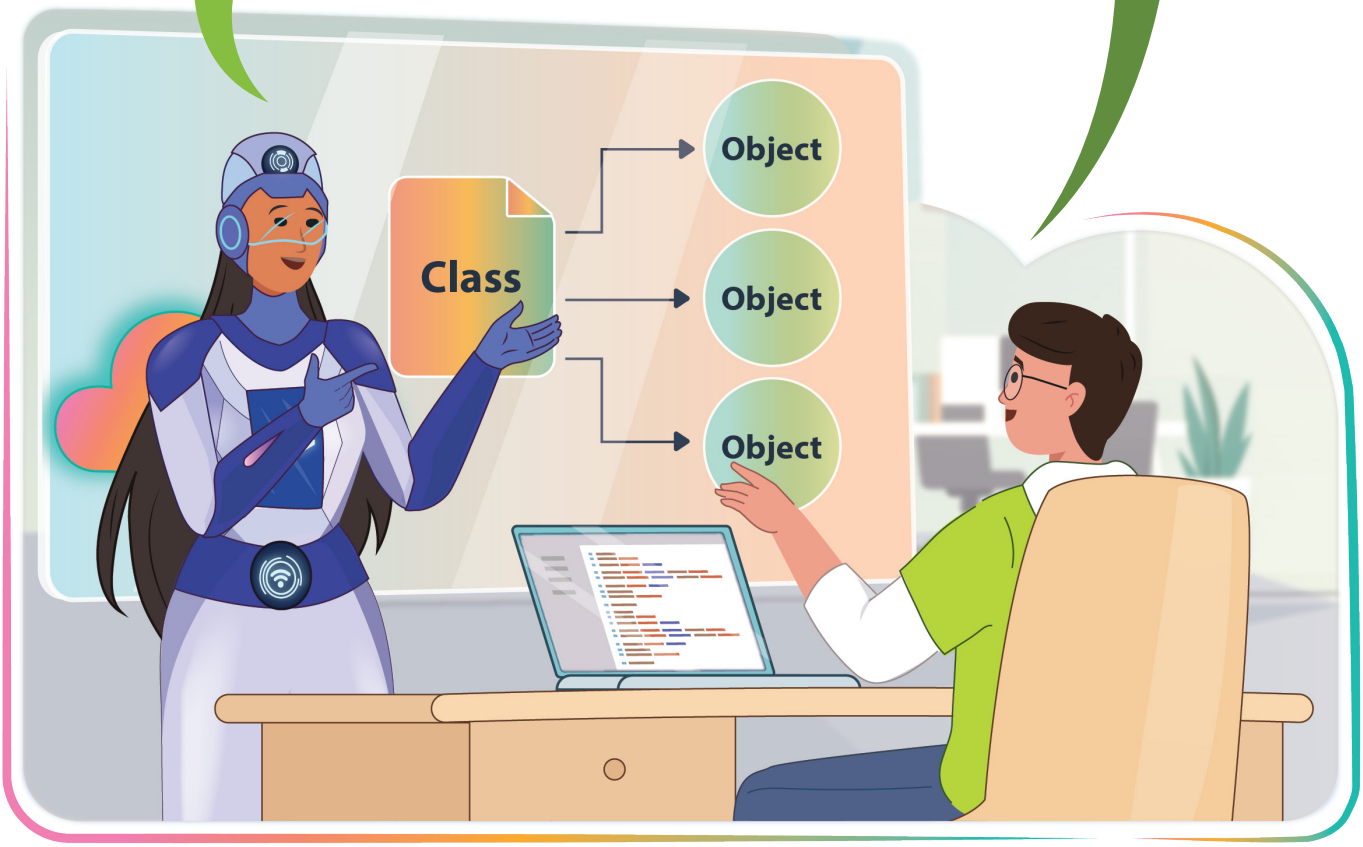
OOP – Class

06

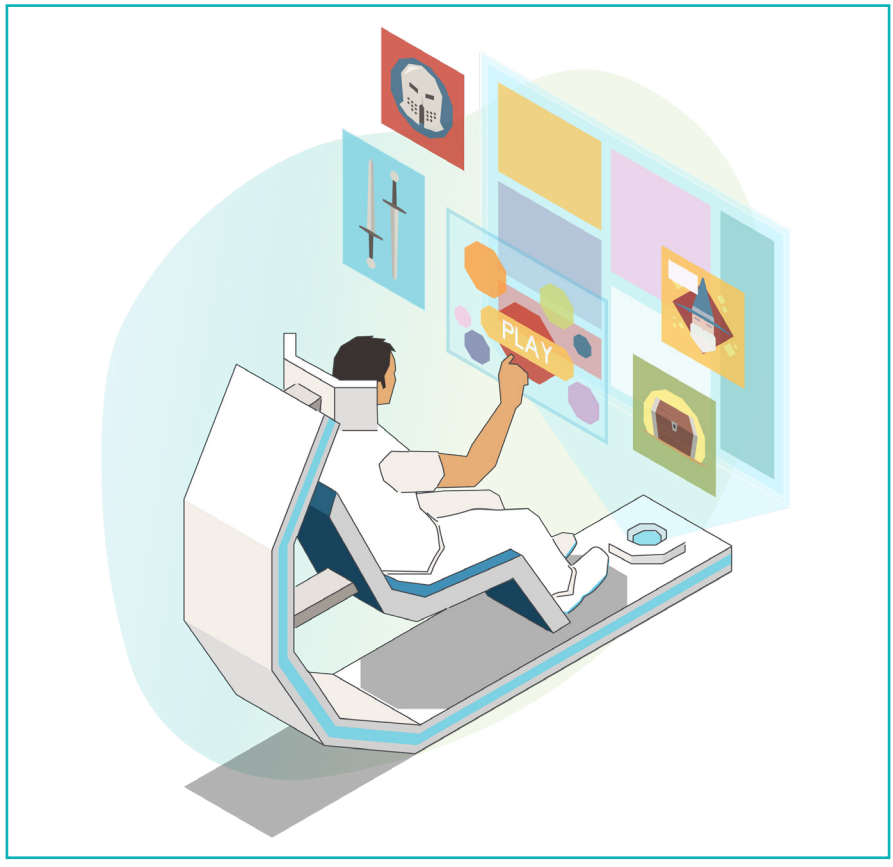
I am familiar with the Objects in programming, however, if I want to create multiple objects of the same structure, the program will become very lengthy. Is there a solution to fix this?

Yes, certainly. You can create a '**Class**' as a blueprint for creating multiple objects from the same class. This will solve your problem, let's explore it.



Introduction

In today's world, high-tech computer systems have enabled us to create simulations that let us experience different activities without doing them. Such systems are called **simulators.**



Simulators are designed to replicate real-world situations or activities so that we can experience them as if we were actually there.. They are used for various purposes, such as entertainment, training, and research. Simulators use advanced technology to create virtual worlds in which we may engage and learn about various events and circumstances.

Click on the QR Code below and try to play the flight simulator game.

https://www.geo-fs.com/geofs.php

Hint: Use arrow keys to maneuver and 1 to 9 keys to accelerate or decelerate.

Simulation is the process of making a model of something in the real world to learn how it behaves or predicts its outcomes in the virtual world. It is similar to creating a simplified version of the real thing, in which we mimic the important parts, components and their interactions. By studying this model, we can gain insights into how the real system works and operates without having to experiment with it directly.

Applications

The flight simulator is a well-known example of a simulator in the aviation industry. It enables individuals to practice flying planes in a realistic environment. It has detailed graphics, accurate controls, and everything else available in a real cockpit. Flight simulators are helpful for pilots to improve their skills as well as provide entertainment for aviation enthusiasts.

Simulators are not limited to just flying and driving. They are also used in a variety of other fields. For instance, medical simulators enable aspiring doctors to practice procedures risk-free, which helps in their training. Engineering simulators help in designing and testing complex systems before building them in real life. There are also space exploration simulators that allow us to explore space and feel what it is like to travel beyond Earth.

Simulation games are also a major part of the entertainment industry where individuals enjoy various simulations for fun. There are simulation games for driving cars, planes and other vehicles. These games allow us to drive on virtual roads and face challenging conditions.

In this chapter, we will explore the most basic elements of simulators or similar games from a coding perspective. We will consider the example of flight simulator games to comprehend how simulation is implemented to experience and interact in the virtual world.

Moving Background Effect

We have encountered this concept in earlier grades, but it is important to review it again from the perspective of creating a simulator of a moving object like a plane.

Moving background effect refers to the technique used to create the illusion that a vehicle or object is moving through an environment by moving the other elements in the background or around the object in opposite directions but keeping the vehicle or object stationary in the screen.

We have to animate the background landscape moving in the opposite direction of the simulated object movement to make it appear moving forward, providing a more realistic and immersive experience. In simulations, such as driving or flight simulators, the moving background effect is crucial for creating a sense of speed, motion, or depth. It gives the user the impression that they are moving through different locations or environments.

Exercise

Answer the questions with one sentence.

1. Name 5 applications that use simulations.

_____

_____

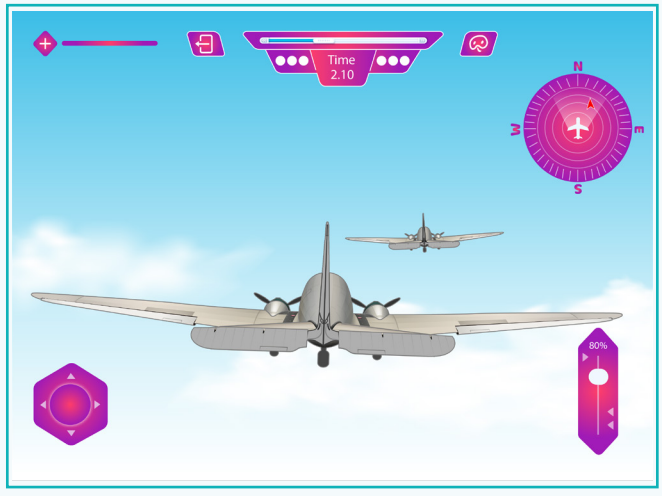
2. Explain the Moving Background effect.

_____

_____

Activity

In this activity, we will create a basic flying plane simulation in which the user can control the speed and tilt of the plane. In this case, the objects that a plane can encounter in the sky are mostly clouds. Therefore, we will create cloud objects that will travel in the backward direction (towards the user) in 3D, while the plane model will remain stationary in one position. This will create the effect of a plane flying ahead.



Let us first create a cloud object using WEBGL and populate it at random locations. We will use multiple spheres to create a cloud shape. Note that creating realistic effects is also possible, but it involves advanced concepts and a lot of coding. Therefore, at this stage, we will only implement basic principles of simulation and we can add details after learning advanced concepts later.

Create a sphere

1. Open the p5js web editor and add 'WEBGL' as the third parameter in the 'createCanvas()' command for 3D mode.

2. Apply light blue colour to the background by changing the values of the background command.

Activity

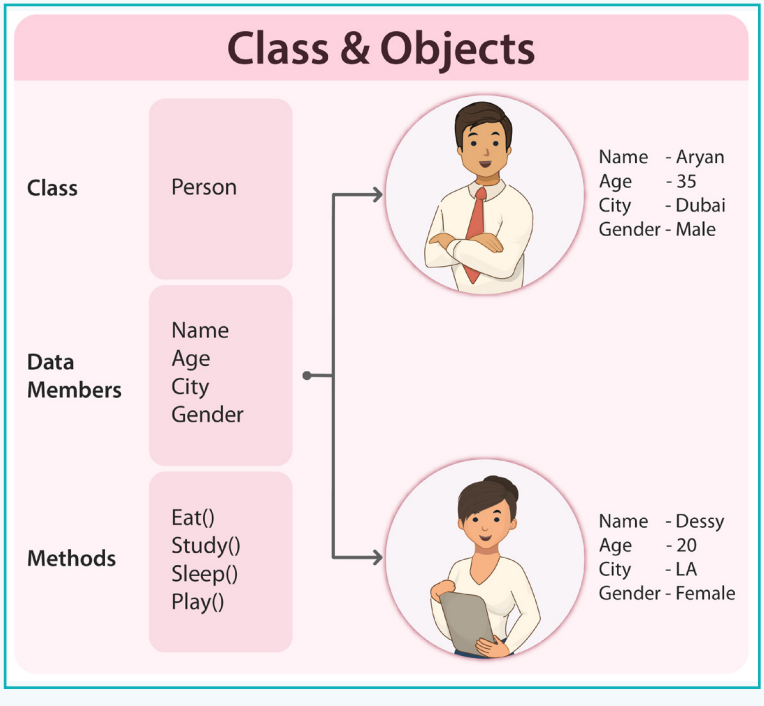
We will create clouds using the concept of OOP. We learned about objects in the previous lesson. Now, we want to create multiple cloud objects with the same properties. Therefore, we will explore the concept of Class in OOP. In JavaScript, classes are introduced in ECMAScript 2015 (ES6) and provide a way to implement the OOP concepts.

Let us recall the concept of OOP before moving into the concept of class.

Object-Oriented Programming (OOP) is a programming paradigm that focusses on organizing code around objects with states (properties) and behaviors (methods). OOP promotes modularity, reusability, and a structured approach to software development.

When we need to create multiple similar objects with the same properties and functionalities, we can create a structure or blueprint. All the objects created from the class will have the same properties with varied values and functionalities. We can think of a class as a definition for a particular type of object. Let us consider the example of a 'Person' class.

The 'Person' class would define the characteristics and actions that each person or object should have. The class would specify the attributes describing a person, such as name, age, and gender. It would also define the methods that a person can perform, such as walk(), eat(), and sleep(). So, if we create 'Aryan' object using the 'Person' class, the object will have its own properties such as name, age, and gender as defined in the class.

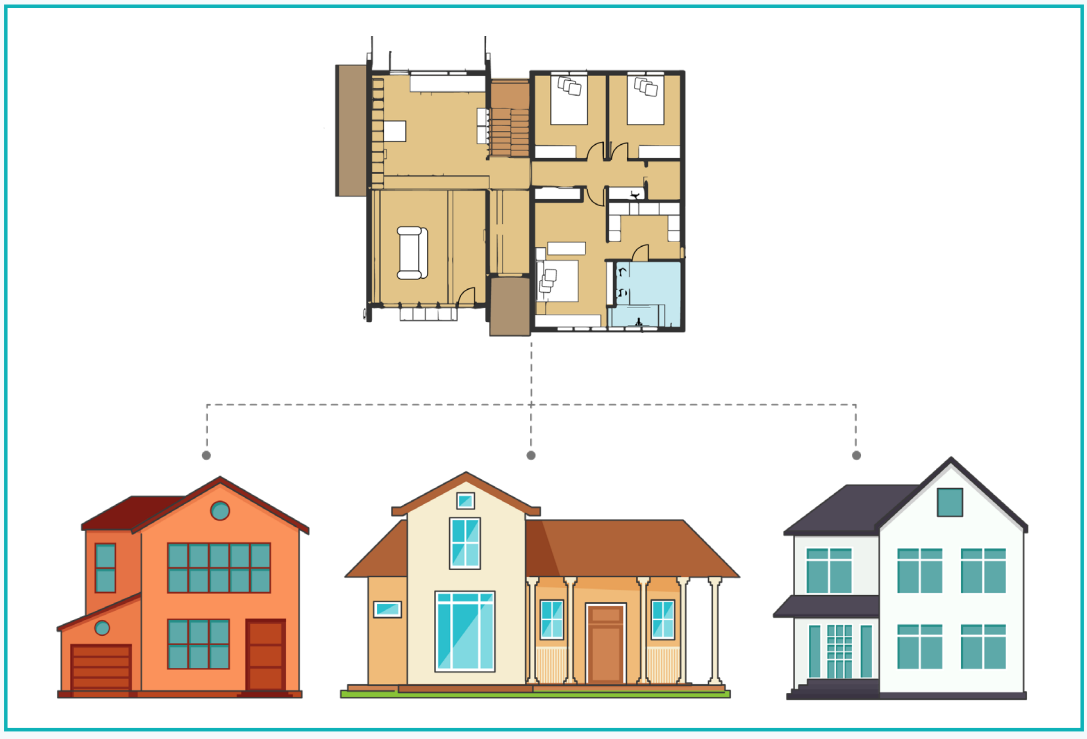


Bytes

ECMAScript 2015 (ES6) is an updated version of JavaScript language that was released in 2015 with significant updates. It introduced various new features and improvements to make JavaScript language more powerful and user-friendly.

Activity

Class - A **class** is like a blueprint or a template that defines the structure and behaviour of the objects that will be created from it. It serves as a guide for creating objects with similar properties and methods.



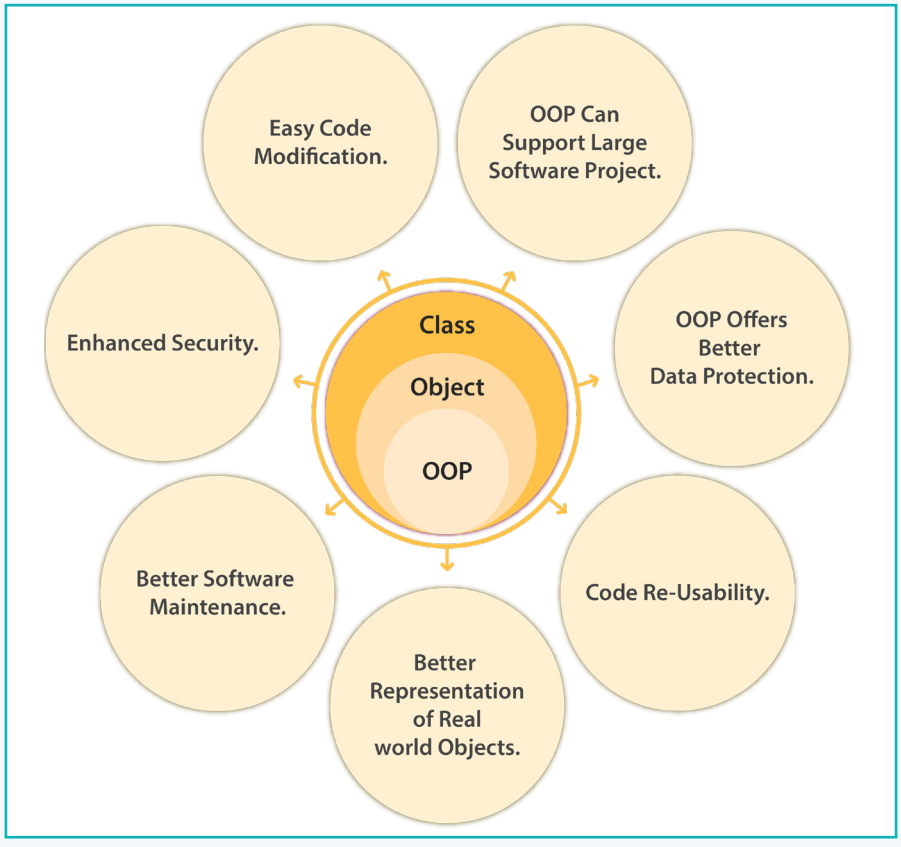
Classes allow us to create multiple objects consistent with their properties and behaviour . Each object may have its unique property values, but they all follow the same structure and behaviour as specified by the class.

Instance of a Class - When we create an object from a class, it is called an 'instance' of that class. For example, we can create instances of the 'Person' class, such as person1 and person2, which represent specific individuals with unique values for properties such as name, age, and gender.

Activity

Advantages

By using classes in JavaScript, we can organize the code into reusable and modular components. Classes provide a way to define the structure, behaviour, and relationships between objects. This makes it easier to maintain and extend complex programs, as well as the modelling of real-world entities in a systematic and consistent manner.



Syntax

To define a class, use the 'class' keyword followed by the class name. According to the naming conventions, class names should start with a capital letter, as PascalCase is recommended for class names. Inside the class, we can declare the properties and methods that will be shared by objects created from the class.

Here is a pseudocode representation of a class:

```
class ClassName {
    // class properties and methods
}
```

Activity

Constructor: The **constructor** is a special method that gets called when an object is created from the class. It initializes the object's properties with the provided values. The constructor typically takes parameters that correspond to the initial values of the object's properties.

```
class ClassName {

    // Constructor
    constructor(arg1, arg2) {
        this.property1 = arg1
        this.property2 = arg2
     }
}
```

this. - The 'this.' keyword is used inside the class before any property as it refers to the current object being referenced or accessed. It allows us to access and work with the properties and methods of that object within a method inside the class.

Exercise

Tick mark ☑ the Correct Answer

3 An object created from a class is called a(an) _____ of that class.

a. Instance ☐ **b.** Property ☐

c. Definition ☐ **d.** Name ☐

4 In a class, the _____ keyword refers to the current object that is being referenced or accessed.

a. int ☐ **b.** this ☐

c. var ☐ **d.** Let ☐

Exercise

Tick mark ☑ the Correct Answer

5 In a class, _____ is a specific method that gets called when an object is created from the class.

a. variable ☐ **b.** render ☐

c. constructor ☐ **d.** this ☐

Activity

With this basic overview of the 'class' concept and related syntax, let us create a class for the clouds.

3 Create a class named 'Cloud'.

4 In the 'Cloud' class, create the 'constructor' method. Initialize positions 'x', 'y', and 'z' to 0 and the radius of the sphere 'r' to '100' using the 'this.' keyword.

sketch.js Saved: just now

```
1  function setup() {
2    createCanvas(400, 400, WEBGL);
3  }
4
5  function draw() {
6    background(150,240,255);
7  }
8
9  class Cloud{
10
11   constructor(){
12     this.x=0;
13     this.y=0;
14     this.z=0;
15     this.r=100;
16   }
17 }
```

Activity

Now, with the properties in place, we can create and add functionality. For example, we can create a functionality that displays a sphere at the given position (x,y,z) of radius 'r'. We can use the 'class' methods for the same.

Methods represent the actions or behaviours that objects can perform.

For example, in the 'Person' class, methods could include actions such as walk(), eat(), and sleep(). These methods define how a person or object would perform those actions.

Let's add method in our code to display the object.

5. Create a method called 'render(){…}' inside the class after the 'constructor' method.

6. Add a 'sphere(this.r)' command to draw a sphere on the canvas. Note that, the 'this' keyword is necessary to use a class property anywhere within the class.

sketch.js Saved: just

```
function setup() {
  createCanvas(400, 400, WEBGL);
}

function draw() {
  background(150,240,255);
}

class Cloud{

  constructor(){
    this.x=0;
    this.y=0;
    this.z=0;
    this.r=100;
  }

  render(){
    sphere(this.r);
  }
}
```

We are not able to see a sphere in the output yet. This is because we just created a class and a method. We need to create an object and then call the 'render' method for that object to be able to show the sphere in the output.

Activity

To create a new object from a class named 'Cloud', we use the 'new' keyword followed by the class name and any required input arguments for the class 'constructor'. The syntax is as follows:

var c = new Cloud(arg1, arg2, ...);

Here,

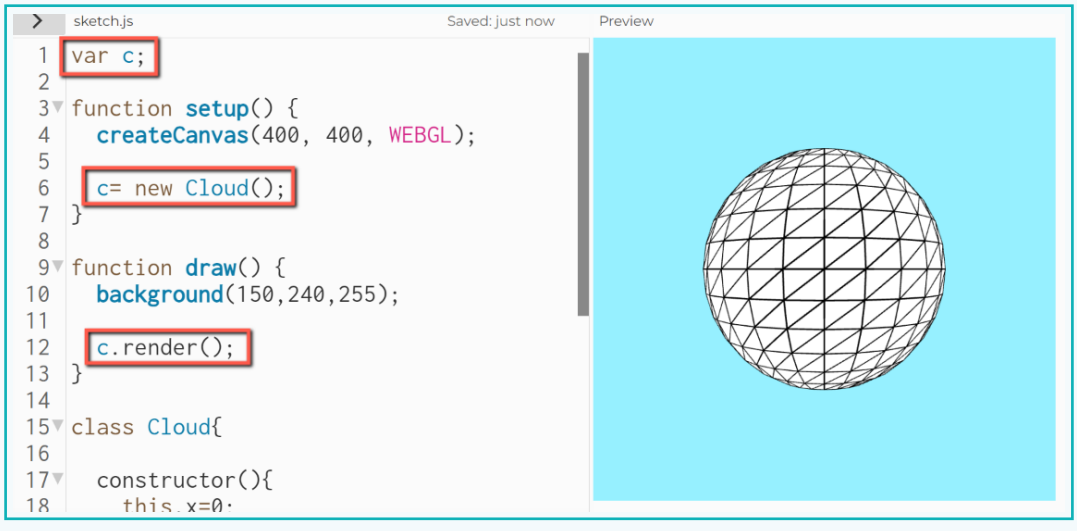
- 'c' is the variable that will hold the newly created object.
- 'Cloud' is the name of the class from which the object is created.
- 'arg1', 'arg2', and so on are optional arguments that may be required as input by the class 'constructor'. These are used to set initial values of the properties of the object. If required by the object design, we should provide the appropriate values as input arguments.

After the object is created, we can access its properties, and invoke its methods using object name followed by the 'dot' notation and method.

objectName.methodName(); // Invoking a method of the object

In our case, no input arguments are necessary. We can simply create a cloud object from a class and invoke the render method to display the sphere.

7. Create a variable 'c' at the beginning of the code.
8. Create a new object from the 'Cloud' class and assign it to the variable 'c' in the 'setup()' function.
9. Use the 'dot (.)' operator in the 'draw' function to invoke the 'render()' method for object 'c'.



Now, we can see a sphere in the output window. Let us add code to update its colour, position, and size in the class.

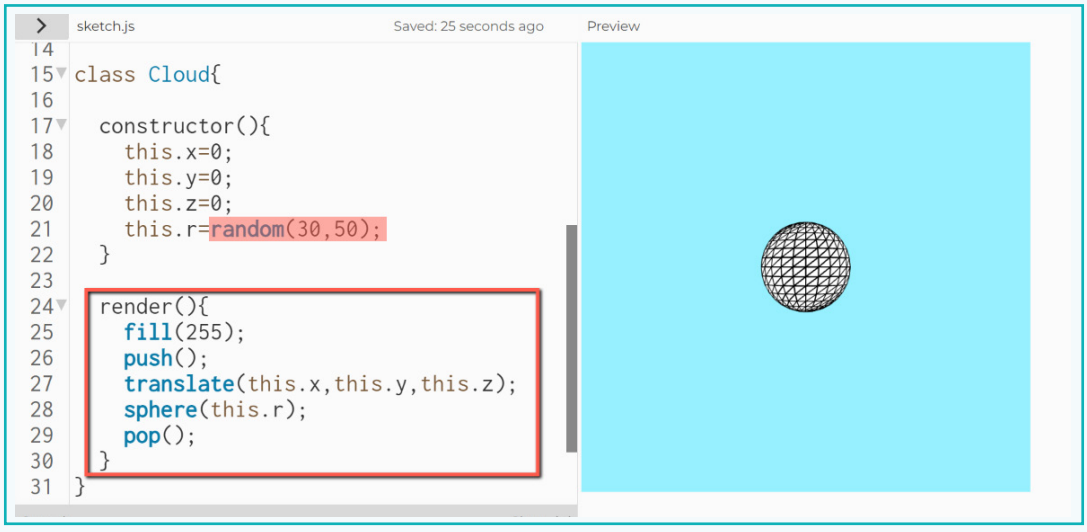
Activity

10. Update the 'this.r' value to 'random(30,50)' so that every sphere contributing to the cloud shape will not be the exact same.

11. Add the 'fill(255)' command to apply the white colour to the sphere.

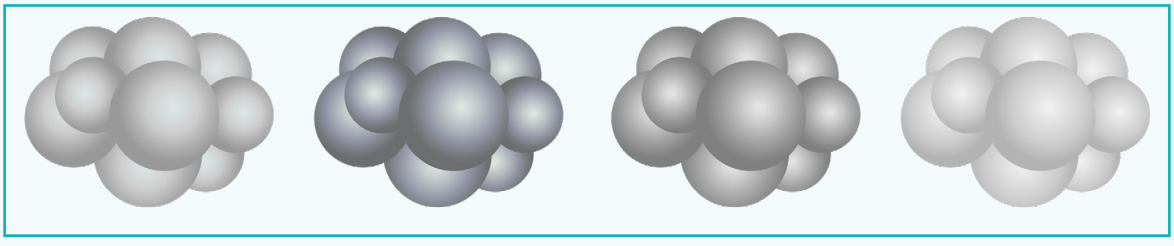
12. Add the 'translate(this.x, this.y, this.z);' command before the 'sphere' command to place the sphere at the x, y, and z positions. Currently, as declared in the 'constructor' method, x, y, and z are equal to '0'.

13. Encapsulate the code to draw and place the sphere in the 'push()' and 'pop()' commands so that the translation or rotation does not affect other objects in the sketch.



Create a Cloud

A cloud usually looks like a bunch of cotton rounds embedded into each other. Though realistic clouds are complex and have varying transparency, we can stack spheres to create simple 3D clouds.



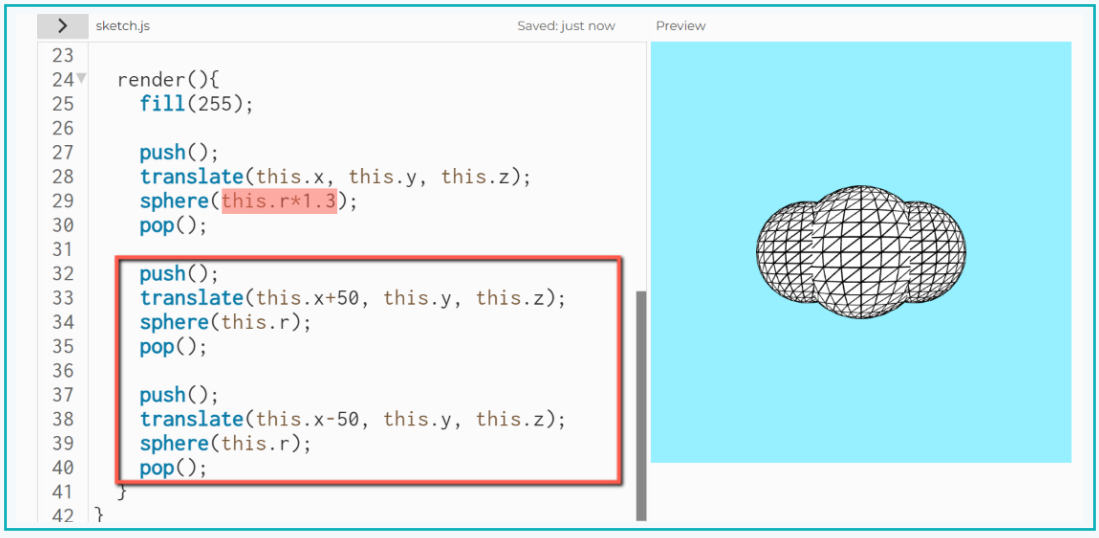
With this base code, let us build a cloud using multiple spheres of different sizes and positions.

Activity

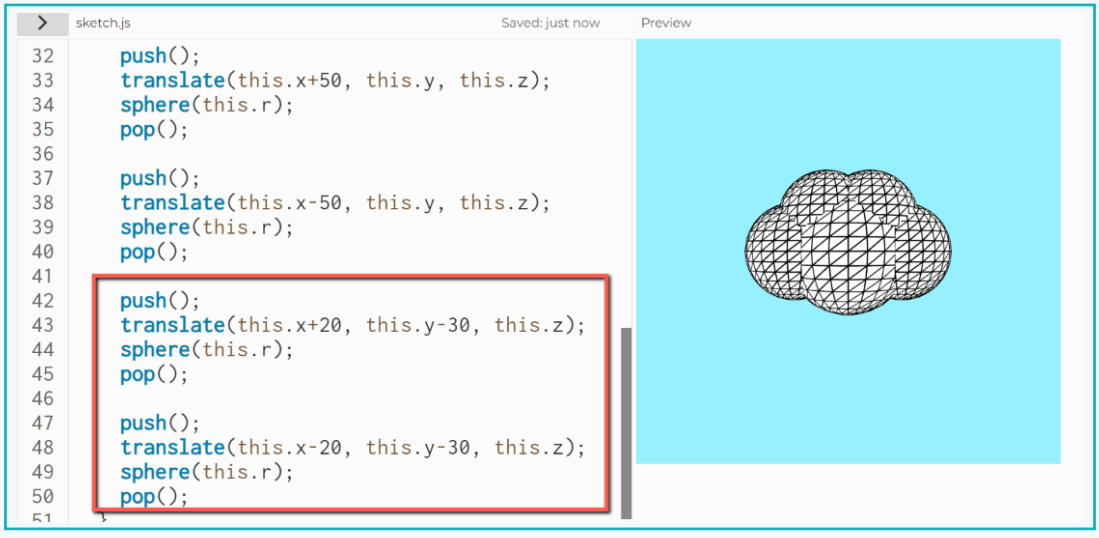
1. Update the input value of the first sphere command to make it 1.3 times the value in the 'r' property.

2. Duplicate the code to create 2 more spheres and update the values of the 'x' position as 'this.x + 50' and 'this.x – 50', respectively.

3. Keep the radius value as 'this.r'.



4. Add two more spheres to create the upper layer of the cloud by displacing along the x and y axes.



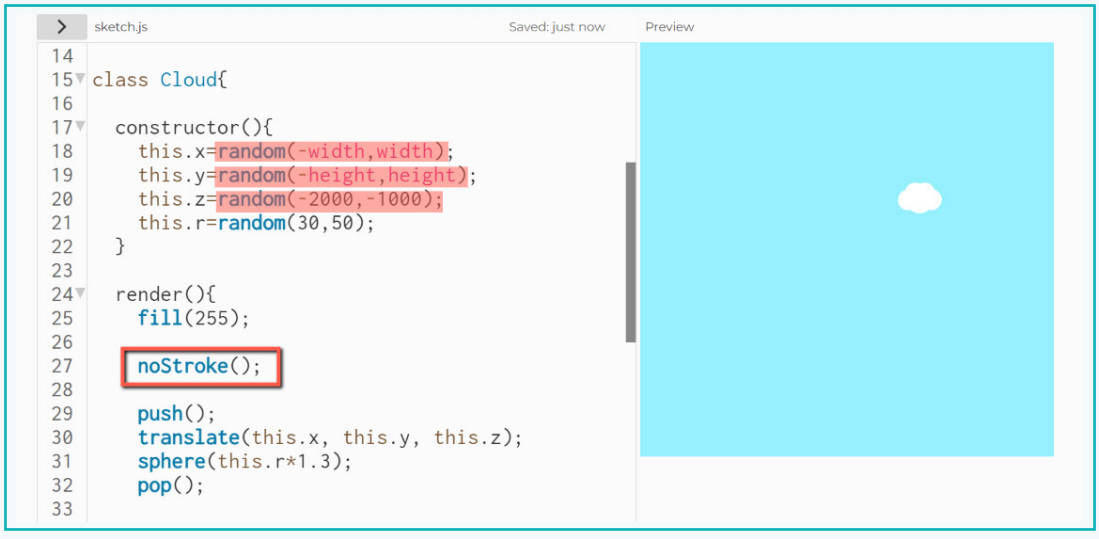
Activity

With this, the design of the basic cloud is complete. Try temporarily adding the 'rotateY(millis()/1000)' command in the draw function to slowly rotate the cloud object we just created and observe its shape in 3D. Now, let us use the cloud we created as intended application.

When a plane moves forward, the cloud appears to move closer from a distance along the z-axis. Hence, clouds should be placed away from the screen along z axis.

5. In the 'constructor' method in the 'Cloud' class, update the initial values of the x, y, and z position properties to have random values between (-width,width), (-height,height), and (-2000, -1000), respectively.

6. Add the 'noStroke()' command in draw before drawing clouds to remove shape lines from the sphere.



A cloud is ready. Now, we can create multiple clouds placed at random locations in 3D space.

Create the Multiple Clouds

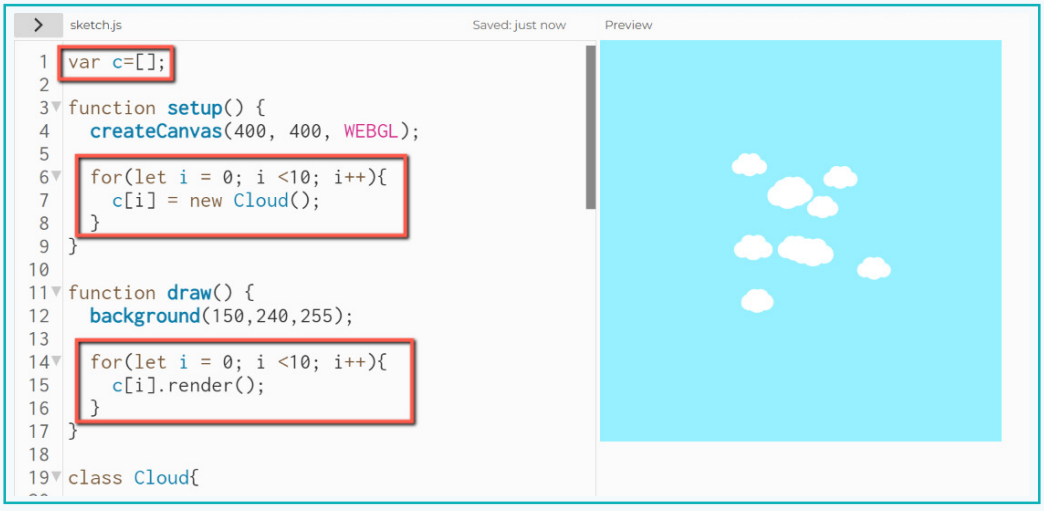
As we need to create multiple objects, we will create an array of objects.

1. Update the existing code to create an array variable 'c'.

2. In setup, update the existing code added to create an object using the 'for' loop to create 10 cloud objects and assign it to each array element of array 'c'.

Activity

3. In draw, update the code added to render the object so that all the objects in the array are rendered using the 'for' loop.

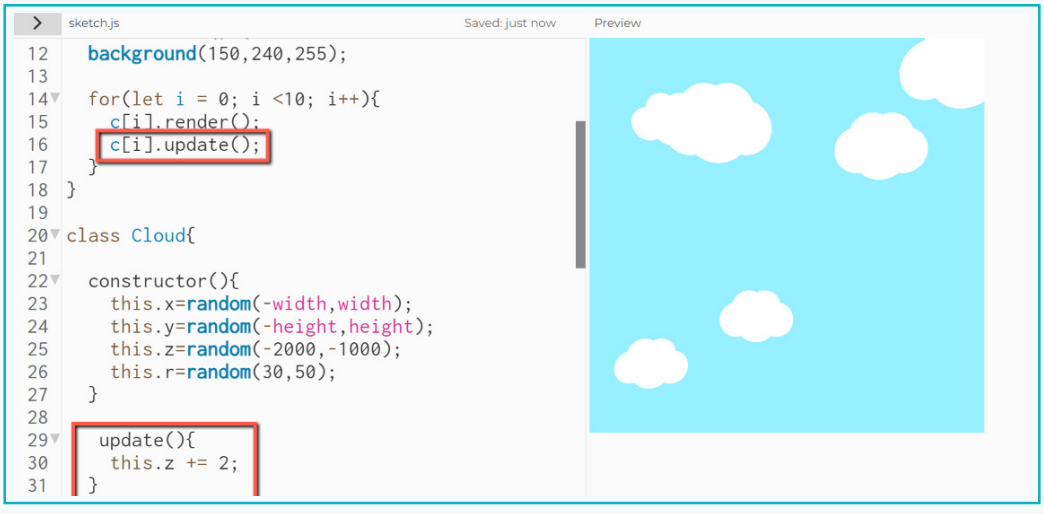


We will be able to see multiple clouds created with this. However, all those look to be placed at a distant location. We can move them by updating their position along the z-axis.

4. Add a new method named 'update()' in the 'Cloud' class.

5. Add code to increment the 'this.z' property value by '2' inside the 'update()' method.

6. Call the 'update' method within the 'for' loop, below the 'render' method in the 'draw' function.



Activity

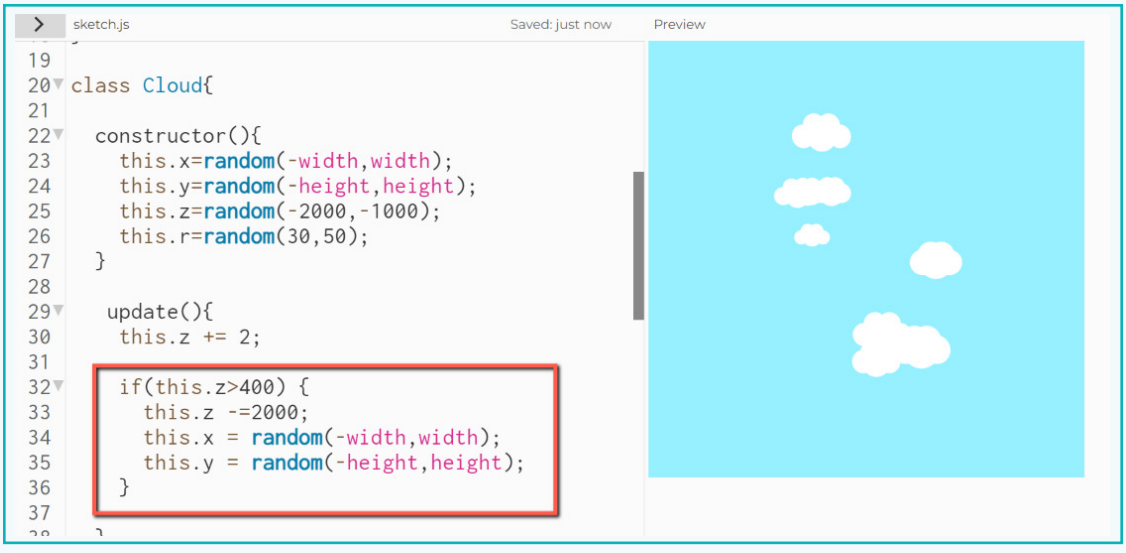
Now, the clouds will appear to move towards the screen from a distance. However, all of them will disappear behind the screen, i.e., we will surpass the clouds. Also, it is not practical to create a huge number of clouds, as they will occupy a large amount of memory.

In such cases, we can repurpose and reuse the vanished clouds by repositioning them far away in the forward direction at random positions. This is a very similar functionality, where if an object goes out of the right edge of the output screen, it appears from the left edge. In the 'z' direction, the screen is positioned at approximately '400' pixels from 'z = 0'.

7. Add the 'if(this.z > 400){…}' condition statement in update() method.

8. If 'true', set the 'z' position to '2000'.

9. Also, set the 'x' and 'y' positions to 'random(-width,width)' and 'random(-height, height)', respectively.



With this, there is a continuous supply of clouds moving towards screen.

Exercise

State if the statements are True or False

6. ‘Methods’ in a class represent the actions or behaviours that objects can perform.

▲ _____

7. It is recommended that the class name be written in CamelCase.

▲ _____

8. A keyword ‘this’ is used to create a new object from a class.

▲ _____

Activity

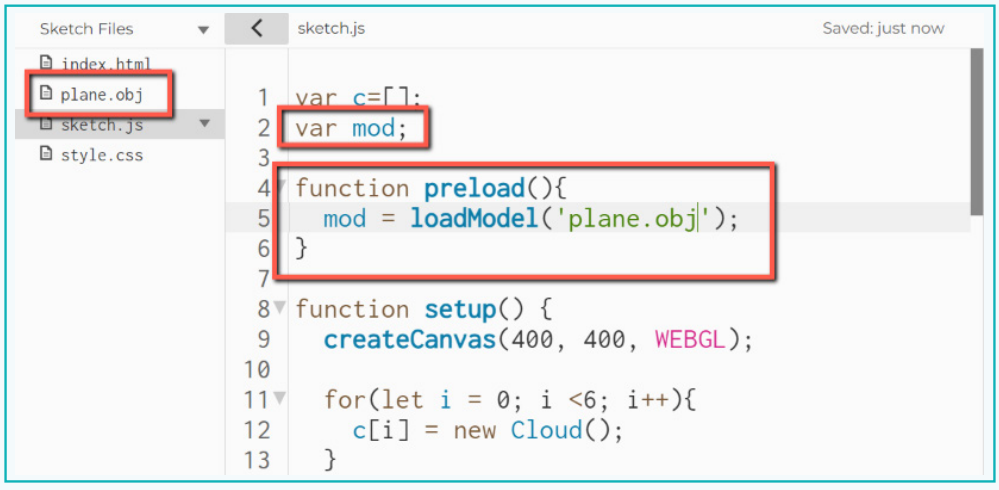
Add a fighter plane

Let us add a 3D plane object in the activity.
Download a 3D model of the plane from the resources using the given QR code.

1. Upload the 3D model of the plane in sketch files.
2. Create a variable named ‘mod’ to load the model in the code.

Activity

3. Add the 'preload()' function and use the 'loadModel('plane.obj')' command to load the 3D model and assign it to the 'mod' variable.



We can also apply texture to the 3D model to make it more realistic.

4. Create a 'texImg' variable at the beginning.

5. Add the image file 'texture.jpg' provided in the resources to the sketch files.

6. Use a 'loadImage('texture.jpg')' command in the 'preload' function and assign it to the 'texImg' variable to load the texture image in the code.

Activity

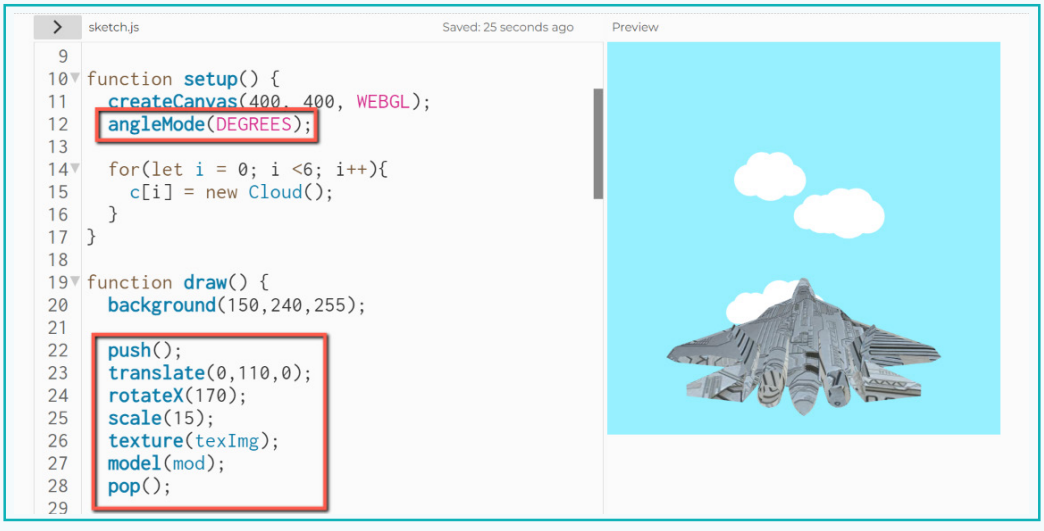
With the model and texture image loaded in the code, we can now display the plane model. Based on the original scale and orientation of the 3D model, it may appear differently in the sketch. We need to scale and rotate it properly to make it suitable for our view.

7. Add a command 'angleMode(DEGREES)' in the setup to ensure all angle values are considered in degrees instead of radians.

8. Add a 'texture(texImg);' command followed by a 'model(mod);' command to display the plane model in draw().

9. Add the 'rotateX(170)' and 'scale(15)' commands before the 'model()' command to enlarge it 15 times to make it visible and rotate at proper angle.

10. Add a 'translate(0,110,0);' command before the earlier commands to place the plane at appropriate position with respect to the 'canvas' window. Encapsulate all commands in the 'push()' and 'pop()' commands.



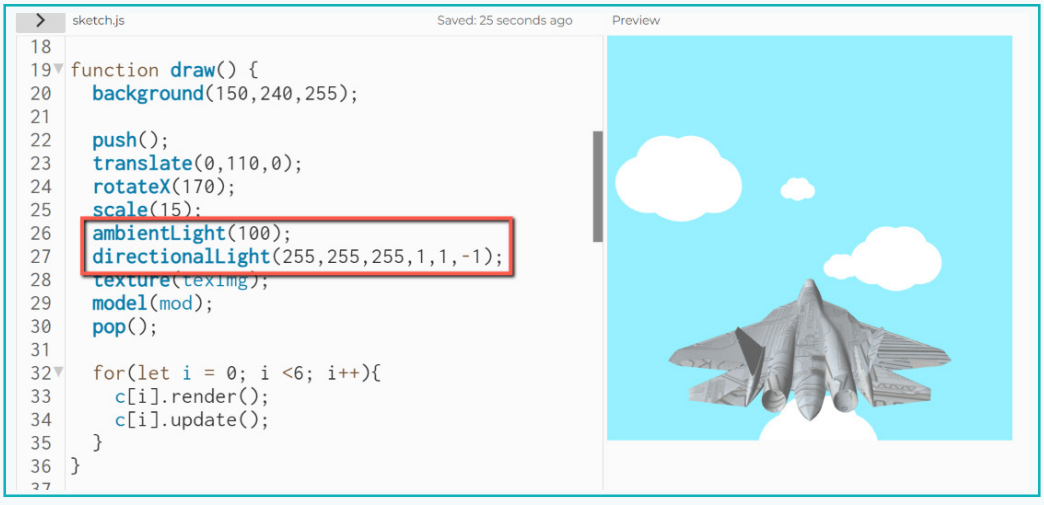
The plane is now clearly visible at the right angle and position, as though it is moving forward. However, without proper lighting, it doesn't look realistic. We can provide adequate lighting to 3D objects in the p5js 'WEBGL' mode. The following commands are helpful in our code.

ambientLight(color, alpha) - This command adds illumination to the scene. Ambient light is a general, evenly distributed light that illuminates all objects equally, contributing to the creation of a base level of illumination and adding overall brightness to the scene. It takes parameters to specify the colour and intensity (alpha) of the ambient light.

Activity

directionalLight(color, direction) - This command creates a directional light source that simulates light coming from a specific direction. Directional light creates shadows and highlights on objects, and their intensity and angle affect their appearance. The 'directionalLight()' function takes parameters for specifying the 'color (r,g,b between 0 to 255)' and 'direction (x,y,z between 1 and -1)' of the light.

11 Add the 'ambientLight(100)' and 'directionalLight(255,255,255,1,1,-1)' commands before the 'model' command to apply lights on the screen.



Now, the plane appears much more realistic. Let us now add the code to change the speed of the plane movement, i.e., cloud movements as well as the angle of the plane.

Activity

Add interactivity and control

1. Declare variables ‘vel’ and ‘ang’ at the beginning. Initialize the variables ‘vel’ and ‘ang’ to ‘5’ and ‘0’ respectively in the setup.

2. Add code to increment ‘this.z’ property variable in the ‘update()’ method with the ‘vel’ variable instead of ‘2’.

```
sketch.js                                   Saved: jus
1  var c=[];
2  var mod;
3  var texImg;
4  var vel;
5  var ang;
6
7  function preload(){
8    mod = loadModel('plane.obj');
9    texImg = loadImage('texture.jpg');
10 }
11
12 function setup() {
13   createCanvas(400, 400, WEBGL);
14   angleMode(DEGREES);
15
16   vel = 5;
17   ang = 0;
18
19   for(let i = 0; i <6; i++){
20     c[i] = new Cloud();
```

```
sketch.js                                   Saved:
43 class Cloud{
44
45   constructor(){
46     this.x=random(-width,width);
47     this.y=random(-height,height);
48     this.z=random(-2000,-1000);
49     this.r=random(30,50);
50   }
51
52    update(){
53     this.z += vel;
54
55     if(this.z>400) {
56       this.z -=2000;
57       this.x = random(-width,width);
58       this.y = random(-height,height);
59     }
60
61   }
```

Now, we can add code to detect the arrow keys pressed and update the values in the ‘vel’ and ‘ang’ variables.

3. Add the ‘conditions’ structure ‘if(){…}’ and ‘else if(…)’ with conditions to check if key is pressed using the ‘keyIsDown()’ command for ‘UP_ARROW’, ‘DOWN_ARROW’, ‘LEFT_ARROW’, and ‘RIGHT_ARROW’ as inputs.

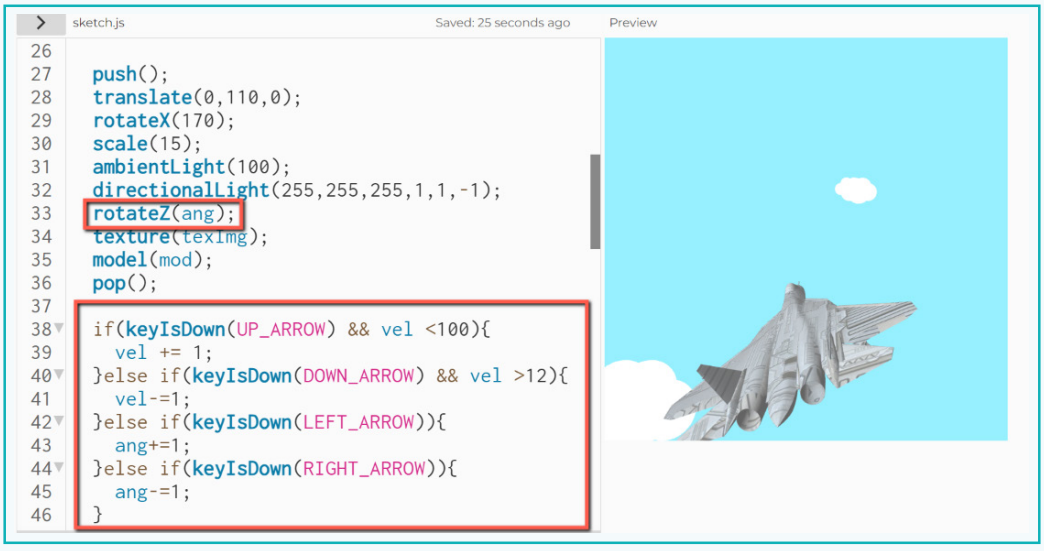
4. When the ‘up’ arrow key is pressed, increment the ‘vel’ variable value by ‘1’. Add another condition as ‘vel<100’ using the ‘&&’ operator to set ‘100’ as its maximum value.

5. When the ‘down’ arrow key is pressed, decrement the ‘vel’ variable value by ‘1’. Add another condition as ‘vel>12’ using the ‘&&’ operator to ensure the minimum value is not ‘less than 12’ to have some minimum speed value

6. Increment and decrement the ‘ang’ variable value if the ‘left’ and ‘right’ arrow keys are pressed, respectively.

Activity

7 Add a ‘rotate(ang);’ command before the ‘model(mod)’ command in the ‘draw’ function to rotate the plane based on the arrow key inputs.



With this, we can now use the arrow keys to change the speed or the angle of the plane, providing interactivity for the simulation. With this, we have completed the simulation of the plane. We can add many features, such as realistic clouds or more complex physical factors, but it adds a significant amount of complexity, which can be attempted as a challenge. Try to make this as a full-fledged simulation game!

Exercise

Answer the question with one sentence.

9 If there is a class named “Mover” as shown

```
class Mover {
   constructor (x,y){
this.x = x;
this.y = y;
                    }
}
```

Create an object ‘ship’ of the class with input as (0,0).

_____

In this lesson, we studied the fundamentals of class and objects to create a basic simulation of a flying plane, with user interaction to vary speed and direction using arrow keys.

6. OOP – Class


'Class' is a very helpful concept in coding for developing applications based on real-world challenges. Like a simulation of the plane, I can create various games on my own and add multiple features of my choice.
Absolutely!

Tech Flash

- **Simulation** : Simulation is the process of making a model of anything in the real world to learn how it behaves or predicts its outcomes in the Virtual/ Digital world.
- **Moving background effect** : Moving background effect in animation refers to the technique used to create the illusion that an object is moving through a changing environment by moving the background elements in opposite directions.
- **Object-Oriented Programming** : Object-Oriented Programming is a programming paradigm that focuses on organizing code around objects with properties and methods.
- **Class** : Class is a template that defines the structure and behaviour of an object that will be created from it with similar properties and methods.
- **Instance of a Class** : Instance of a Class is an object created from the class.
- **PascalCase** : PascalCase is a naming convention in which each word within an identifier starts with an uppercase letter and contains no spaces or underscores.
- **Constructor** : Constructor is a specific method that is invoked when an object is created from the class, and it initializes the object's properties with the provided values.
- **this** : this. keyword in class refers to the current object that is being referenced or accessed.
- **Method** : Method in class represents the actions or behaviours that an object created from the class can perform.